Параллельное программирование на OpenMP

Семинар 1. От MPI к общей памяти

Основы параллельного программирования

Правила работы

- Пользоваться можно **любыми** источниками и инструментами во всех ДЗ
- За каждую строку отвечаете **вы**: на защите попросят объяснить и изменить код без посторонней помощи
- Все числа в отчёте — **с вашей машины**, сырые данные в `results/*.csv`
- В каждом ДЗ файл `SOURCES.md`: чем пользовались и **где источник ошибся**
- Контрольная и защита — с закрытыми ноутбуками (ноутбук только для запуска своего кода)
- Заимствование не штрафуется. Штрафуется только неспособность объяснить сданное

> Написать директиву легко. Доказать, что код верен и быстр, — вот навык.

Что нельзя взять готовым

- Предсказать, как поведёт себя программа на **вашем** ноутбуке
- Объяснить, почему на 6 потоках ускорение упало
- Гарантировать отсутствие гонки: код компилируется, ответ «почти правильный»
- Подтвердить совет «так быстрее» — это гипотеза, её проверяет замер

> 🔍 **Разбор чужого кода**
>
> В конце каждого занятия — разбор чужого кода: готового решения из интернета или от однокурсника. Я показываю ошибку, доказываю её инструментом и исправляю.

Три модели параллелизма

MPI · CUDA · OpenMP

Вы уже знаете две модели

❓ Вопрос

MPI-программа работает на 4 процессах. Где лежат данные? Как процесс 2 узнаёт промежуточный результат процесса 3?

Вы уже знаете две модели

❓ Вопрос

MPI-программа работает на 4 процессах. Где лежат данные? Как процесс 2 узнаёт промежуточный результат процесса 3?

✅ Ответ

У каждого процесса своя память. Узнать можно только сообщением: `MPI_Send/Recv`, `MPI_Reduce`.

	MPI	CUDA	OpenMP
Кто работает	процессы	тысячи нитей GPU	потоки ОС (≈ ядра)
Память	своя у каждого	хост + GPU	**общая**
Обмен	сообщения	`cudaMemcpy`	просто читаем/пишем
Подход	переписать программу	написать ядро	**добавить директивы**
Масштаб	тысячи узлов	один GPU	один узел

Словарь соответствий

MPI / CUDA	OpenMP
`MPI_Comm_rank` / `threadIdx.x`	`omp_get_thread_num()`
`MPI_Comm_size` / `blockDim.x`	`omp_get_num_threads()`
`MPI_Reduce`, `MPI_Allreduce`	`reduction(+:s)`
`MPI_Barrier` / `__syncthreads()`	`#pragma omp barrier`
`atomicAdd`	`#pragma omp atomic`
`mpirun -np 4`	`OMP_NUM_THREADS=4`
запуск ядра `<<<grid, block>>>`	`#pragma omp parallel`

Где аналогия ломается

MPI → OpenMP

- В MPI нельзя испортить чужие данные: их у вас **нет**
- В OpenMP всё **общее по умолчанию**, поэтому главная ошибка — **гонка данных**

CUDA → OpenMP

- CUDA: тысячи лёгких нитей, у каждой крошечная работа
- OpenMP: потоков ≈ ядер, у каждого **большая порция**
- Копировать данные не нужно, но потоки **делят кэш и полосу памяти**

Главное преимущество: инкрементальность

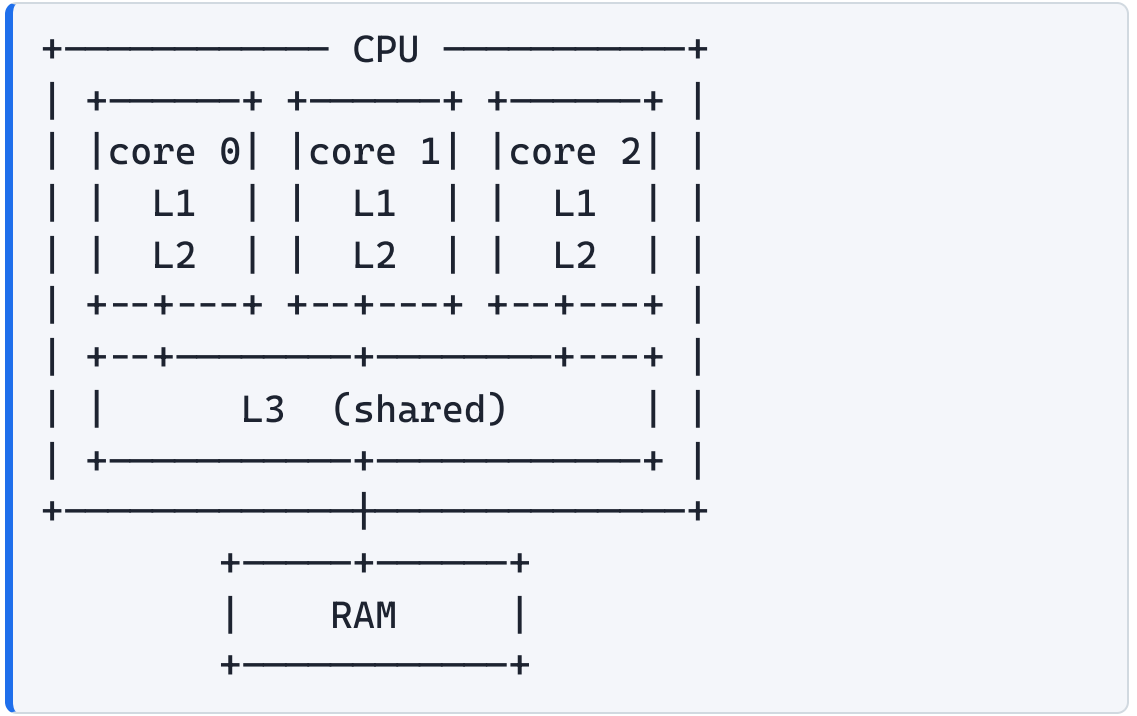
```
double sum = 0.0;
#pragma omp parallel for reduction(+:sum)   /* одна строка */
for (long i = 0; i < n; i++) {
    double x = (i + 0.5) * h;
    sum += 4.0 / (1.0 + x * x);
}
```

- Есть отлаженный последовательный код модели — параллелим **горячие циклы по одному**
- Без `-fopenmp` прагмы игнорируются: та же программа остаётся последовательной
- Сравнение с последовательной версией — главный тест корректности

Что внутри процессора вашего ноутбука

Многоядерный CPU

- **Ядро** — независимый исполнитель
- **SMT / Hyper-Threading**: 2 логических процессора на ядро делят его блоки. Для численного кода +0–20 %
- **Кэши**: L1, L2 у ядра, L3 общий. Обмен идёт **линиями по 64 байта**
- **Память** общая для всех ядер, её полоса тоже



Ноутбук — не сервер

- **Гибридные ядра**: Intel 12-го поколения и новее (P + E), Apple M1–M4 (performance + efficiency). E-ядра медленнее
- **Турбо**: 1 поток работает на 4,8 ГГц, 8 потоков — на 3,5 ГГц
- **Нагрев и питание**: от батареи частоты ниже, при долгой нагрузке ноутбук «троттлит»

> ❓ **Вопрос**
>
> 6 P-ядер с HT + 8 E-ядер. Сколько логических процессоров увидит ОС? Какое ускорение ждать на 20 потоках?

Ноутбук — не сервер

- **Гибридные ядра**: Intel 12-го поколения и новее (P + E), Apple M1–M4 (performance + efficiency). E-ядра медленнее
- **Турбо**: 1 поток работает на 4,8 ГГц, 8 потоков — на 3,5 ГГц
- **Нагрев и питание**: от батареи частоты ниже, при долгой нагрузке ноутбук «троттлит»

❓ Вопрос

6 P-ядер с HT + 8 E-ядер. Сколько логических процессоров увидит ОС? Какое ускорение ждать на 20 потоках?

✅ Ответ

6·2 + 8 = **20** логических. Ускорение заметно меньше 20, на численной задаче скорее 7–10: SMT почти не помогает, E-ядра медленнее, частота падает.

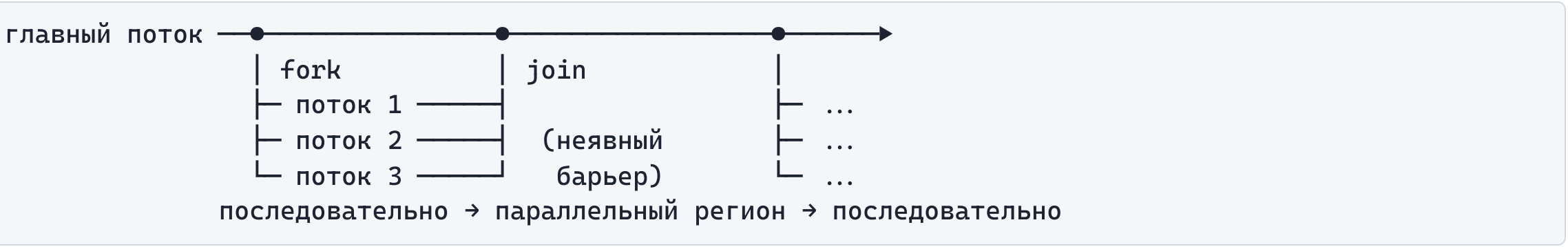
Как узнать своё железо

ОС	Команда
Linux / WSL	`lscpu`, `nproc`
macOS	`sysctl -n machdep.cpu.brand_string` `sysctl hw.perflevel0.physicalcpu hw.perflevel1.physicalcpu`
Windows	Диспетчер задач → Производительность → ЦП

Модель, физические ядра, логические процессоры, P/E — в Д30 это станет **паспортом стенда** `SYSTEM.md`. Собирает его скрипт `tools/sysinfo.sh` или `sysinfo.ps1`.

Модель fork-join

Fork-join



- Программа стартует **одним** потоком (master, номер 0)
- `#pragma omp parallel` порождает **команду** потоков; каждый выполняет блок целиком
- В конце региона **неявный барьер**: дальше идёт только главный поток
- Потоки берутся из пула, но вход в регион всё равно стоит микросекунды

Первая программа

```c
#include <stdio.h>
#include <omp.h>

int main(void)
{
    printf("До региона: потоков %d\n", omp_get_num_threads());
    #pragma omp parallel
    {
        int id = omp_get_thread_num();   /* у каждого потока своя копия */
        printf("Привет от потока %d из %d\n", id, omp_get_num_threads());
    }                                    /* неявный барьер */
    printf("После региона: потоков %d\n", omp_get_num_threads());
}
```

❓ Вопрос

Что напечатает `OMP_NUM_THREADS=4 ./hello`? В каком порядке?

Первая программа

```c
#include <stdio.h>
#include <omp.h>

int main(void)
{
    printf("До региона: потоков %d\n", omp_get_num_threads());
    #pragma omp parallel
    {
        int id = omp_get_thread_num();   /* у каждого потока своя копия */
        printf("Привет от потока %d из %d\n", id, omp_get_num_threads());
    }                                    /* неявный барьер */
    printf("После региона: потоков %d\n", omp_get_num_threads());
}
```

❓ Вопрос

Что напечатает `OMP_NUM_THREADS=4 ./hello`? В каком порядке?

✅ Ответ

4 строки в **произвольном** порядке, он меняется от запуска к запуску. До и после региона — 1 поток.

Сколько потоков?

Приоритет растёт слева направо: по умолчанию (= число логических процессоров) → `OMP_NUM_THREADS=8` → `omp_set_num_threads(8)` → клауза `num_threads(8)` у директивы

Функция	Что возвращает
`omp_get_thread_num()`	номер потока в команде, 0…n−1
`omp_get_num_threads()`	размер текущей команды (**вне региона = 1**)
`omp_get_max_threads()`	сколько потоков будет в следующем регионе
`omp_get_num_procs()`	число логических процессоров
`omp_get_wtime()`	настенное время, с

> `omp_get_num_threads()` до региона почти всегда ошибка: нужен `omp_get_max_threads()`.

Сборка на вашей ОС

Система	Команда
Linux, WSL, MSYS2	`gcc -O2 -fopenmp prog.c -o prog`
Linux (clang)	`clang -O2 -fopenmp prog.c -o prog` (нужна libomp)
macOS + Homebrew gcc	`gcc-NN -O2 -fopenmp prog.c -o prog`
macOS, Apple clang	`clang -Xpreprocessor -fopenmp -lomp ...` (+ пути brew libomp)

> Visual Studio `/openmp` — это OpenMP 2.0 (2002 год): нет `collapse`, задач, `min/max`-редукций. Для курса не подходит: используйте WSL2 или MSYS2. Подробно — `docs/02-okruzhenie.md`.

Макрос `_OPENMP` и версии

```c
#ifdef _OPENMP
    printf("OpenMP %d\n", _OPENMP);   /* дата версии стандарта */
#else
    printf("собрано без OpenMP\n");
#endif
```

`_OPENMP`	Версия	Что появилось
200805	3.0	задачи (`task`)
201307	4.0	`simd`, `target`, `depend`
201511	4.5	редукции по массивам, `taskloop`
201811	5.0	`loop`, улучшенные задачи и offload
202011	5.1	`masked`, `error`

gcc 13 сообщает 201511 (полностью реализован 4.5, многое из 5.x тоже есть), clang 18 — 202011.

Кто видит какие данные

Области видимости

- Объявлено **до** региона → `shared`: одна копия на всех
- Объявлено **внутри** региона → `private`: своя копия у каждого потока
- `private(x)` — своя копия, **не инициализирована**
- `firstprivate(x)` — своя копия, **инициализирована** значением до региона
- После региона приватные копии исчезают, оригинал не меняется

```
int a = 10, b = 20, c = 30;
#pragma omp parallel private(b) firstprivate(c) shared(a)
{ ... }
```

`default(none)` заставляет явно указать область каждой переменной. Это лучший способ ловить ошибки, в том числе в чужом коде (C2).

Что напечатает `scope.c`?

```
int a = 10, b = 20, c = 30;
#pragma omp parallel private(b) firstprivate(c) shared(a) num_threads(4)
{
    int id = omp_get_thread_num();
    b = id;
    c += id;
    if (id == 0) a = 99;
    #pragma omp barrier
    #pragma omp critical
    printf("поток %d: a=%d b=%d c=%d\n", id, a, b, c);
}
printf("после: a=%d b=%d c=%d\n", a, b, c);
```

Что напечатает `scope.c`?

```
int a = 10, b = 20, c = 30;
#pragma omp parallel private(b) firstprivate(c) shared(a) num_threads(4)
{
    int id = omp_get_thread_num();
    b = id;
    c += id;
    if (id == 0) a = 99;
    #pragma omp barrier
    #pragma omp critical
    printf("поток %d: a=%d b=%d c=%d\n", id, a, b, c);
}
printf("после: a=%d b=%d c=%d\n", a, b, c);
```

✅ **Ответ**

Поток `id`: `a=99 b=id c=30+id`. После региона: `a=99 b=20 c=30` — приватные копии исчезли, оригиналы не тронуты.

Правило хорошего стиля

Опасно (стиль C89)

```
double x, f;
int i;
#pragma omp parallel
{
    for (i = id; i < n; i += nt) {
        x = (i + 0.5) * h;
        f = 4.0 / (1.0 + x * x);
        ...
```

`i`, `x`, `f` — **общие**!

Безопасно

```
#pragma omp parallel
{
    for (long i = id; i < n; i += nt) {
        double x = (i + 0.5) * h;
        double f = 4.0 / (1.0 + x * x);
        ...
```

Всё объявлено внутри — всё приватно

Объявляйте переменные **как можно позже**, в самом внутреннем блоке.

Сквозной пример: вычисляем π

$$\pi = \int_0^1 \frac{4}{1+x^2}\,dx$$

Последовательная версия

```
double h = 1.0 / n, sum = 0.0;
double t0 = omp_get_wtime();
for (long i = 0; i < n; i++) {
    double x = (i + 0.5) * h;          /* средние прямоугольники */
    sum += 4.0 / (1.0 + x * x);
}
double pi = sum * h;
double t = omp_get_wtime() - t0;
```

`code/pi_serial.c`, n = $4 \cdot 10^8$, ошибка ≈ 10^{-13}, время ≈ 0,5 с

Задача: распараллелить «как в MPI» — каждый поток берёт свою часть индексов.

Шаг 1. SPMD-разбиение

```
#pragma omp parallel
{
    int id = omp_get_thread_num();       /* аналог MPI rank */
    int nt = omp_get_num_threads();      /* аналог MPI size */
    for (long i = id; i < n; i += nt) { /* циклическое разбиение */
        double x = (i + 0.5) * h;
        sum += 4.0 / (1.0 + x * x);       /* sum — общая! */
    }
}
```

❓ Вопрос

Что получится на 2 и 4 потоках?

▶ Демонстрация

`make run` — версия `pi_spmd_race` на 1, 2, 4, 8 потоках. Ответ на следующем слайде.

Результат зависит от компилятора

Компилятор	1 поток	2 потока	4 потока	время на 2 потоках
gcc 13 -O2	3.14159…	1.57079…	0.78539…	быстрее
clang 18 -O2	3.14159…	1.57123…	0.78540…	медленнее, чем 1 поток

- **gcc** держит `sum` в регистре и записывает один раз: выигрывает последний записавший
- **clang** читает и пишет общую ячейку на каждой итерации: обновления теряются, ядра дерутся за кэш-линию

Гонка данных: два потока обращаются к одной ячейке, хотя бы одно обращение — запись, синхронизации нет. Это **неопределённое поведение**: результат может быть любым.

Почему теряются обновления

`sum += f` — это три операции:

```
поток 0                    поток 1                 sum в памяти
загрузить sum → r0 (=5)                              5
                           загрузить sum → r1 (=5)   5
r0 = r0 + 2   (=7)                                   5
                           r1 = r1 + 3   (=8)        5
записать r0 → sum                                    7
                           записать r1 → sum         8   ← «+2» потеряно
```

Ни компилятор, ни рантайм не предупредят. Ответ «почти правильный» — худший вариант: его легко не заметить.

Шаг 2. Ловим гонку инструментом

```
clang -O1 -g -fopenmp -fsanitize=thread pi_spmd_race.c -o race
TSAN_OPTIONS=ignore_noninstrumented_modules=1 \
OMP_TOOL_LIBRARIES=/usr/lib/llvm-18/lib/libarcher.so \
OMP_NUM_THREADS=2 ./race 100000
```

```
WARNING: ThreadSanitizer: data race
  Write of size 8 at 0x7ffe432e76dc by thread T1:
    #0 main.omp_outlined_debug__ pi_spmd_race.c:25
  Previous write of size 8 at 0x7ffe432e76dc by main thread:
    #0 main.omp_outlined_debug__ pi_spmd_race.c:25
```

- **ThreadSanitizer** + **Archer** (модуль, который объясняет TSan семантику OpenMP)
- В вашем репозитории это делает `make tsan` и CI на каждый push

Шаг 3. Критическая секция

```
for (long i = id; i < n; i += nt) {
    double x = (i + 0.5) * h;
    double f = 4.0 / (1.0 + x * x);
    #pragma omp critical
    sum += f;                  /* в секции одновременно один поток */
}
```

- Ответ **верный**
- n = $2 \cdot 10^7$: последовательно 0,03 с, `critical` на 2 потоках — **5,5 с** (наш замер на 2 ядрах)
- Каждая итерация стоит в очереди. Аналогия: `MPI_Send` на каждое слагаемое

Шаг 4. Массив частичных сумм

```
double partial[MAXT] = {0.0};
#pragma omp parallel
{
    int id = omp_get_thread_num(), nt = omp_get_num_threads();
    if (id == 0) nthreads = nt;           /* узнать можно только внутри */
    for (long i = id; i < n; i += nt) {  ...  partial[id] += f; }
}
for (int k = 0; k < nthreads; k++) sum += partial[k];
```

- Как `MPI_Gather` + сумма на корне. Гонки нет, ответ верный
- Но `partial[0..7]` лежат в **одной кэш-линии** 64 байта, и ядра перетягивают её друг у друга
- Это **false sharing** (ложное разделение) — подробно в C4

Шаг 5. Локальная сумма + `atomic`

```
#pragma omp parallel
{
    int id = omp_get_thread_num(), nt = omp_get_num_threads();
    double local = 0.0;                    /* private: объявлена внутри */
    for (long i = id; i < n; i += nt) {
        double x = (i + 0.5) * h;
        local += 4.0 / (1.0 + x * x);
    }
    #pragma omp atomic
    sum += local;                           /* одна атомарная операция на поток */
}
```

❓ Вопрос

Сколько синхронизаций на 8 потоках? А в версии с `critical` при $n = 4 \cdot 10^8$?

Шаг 5. Локальная сумма + `atomic`

```
#pragma omp parallel
{
    int id = omp_get_thread_num(), nt = omp_get_num_threads();
    double local = 0.0;                    /* private: объявлена внутри */
    for (long i = id; i < n; i += nt) {
        double x = (i + 0.5) * h;
        local += 4.0 / (1.0 + x * x);
    }
    #pragma omp atomic
    sum += local;                           /* одна атомарная операция на поток */
}
```

❓ Вопрос

Сколько синхронизаций на 8 потоках? А в версии с `critical` при $n = 4 \cdot 10^8$?

✅ Ответ

8 атомарных операций против $4 \cdot 10^8$ входов в критическую секцию. Это правильный шаблон, ровно его генерирует `reduction(+:sum)` (C2).

Сводка по π

Версия	Верно?	Быстро?	Почему
общая `sum`	**нет**	как повезёт	гонка данных, UB
`critical` на итерацию	**да**	**нет**	очередь на каждое сложение
массив `partial[id]`	**да**	средне	false sharing
локальная + `atomic`	**да**	**да**	синхронизация раз на поток

Общий принцип: **сначала считаем приватно, потом редко объединяем**. Так же, как в MPI: локальные вычисления + одна коллективная операция.

Как мерить время

Правильно

- `omp_get_wtime()` — настенное время
- мерить только расчёт
- ≥ 5 повторов, **медиана**
- первый запуск — прогрев
- ноутбук **на зарядке**, режим производительности
- p = 1, 2, 4, … до 2 × логических ядер

Неправильно

- `clock()`: на Linux и macOS это **суммарное процессорное время всех потоков**

```
$ OMP_NUM_THREADS=2 ./timing
omp_get_wtime: 0.295 c
clock():       0.570 c
```

Ускорение и эффективность

- **Ускорение** $S(p) = T(1)/T(p)$
- **Эффективность** $E(p) = S(p)/p$
- $T(1)$ — время на одном потоке. Всегда пишите, что именно взяли за $T(1)$: параллельную программу на 1 потоке или лучшую последовательную

Почему на ноутбуке $S(p) < p$ даже для π, где нет ни обменов, ни памяти:
турбо-частота падает с ростом числа активных ядер; E-ядра медленнее; SMT не удваивает вычислительные блоки.

Замер: прогноз и реальность

▶ Демонстрация

```
python3 ../../../tools/bench.py --cmd "./pi_local_atomic {n}" \
    --threads 1,2,4,8,16,24,32,48,64 --set n=400000000 --repeat 5 --out t.csv
python3 ../../../tools/plot.py t.csv --kind speedup --table
```

p	1	2	4	8	16	24	32	64
прогноз S(p)	1	2	4	8	16	24	32	64
T(p), медиана, мс	302	153	77	42	39	34	28	28
S(p)	1	2,0	3,9	7,1	7,7	9,0	10,8	10,9

Наш замер: i9-13900K — 8 P-ядер с HT и 16 E-ядер, всего 24 ядра и 32 логических процессора (WSL, gcc 13). До 8 потоков почти идеально. На 16 рост почти останавливается: доли работы равные, часть потоков попадает на медленные E-ядра или делит P-ядро с соседом по HT, а на барьере все ждут самого медленного. После 32 — плато: потоков больше, чем процессоров.

Почему S(p) < p даже для π

❓ Вопрос

У двух ноутбуков одинаковое число ядер, а ускорения на π разные. Почему?

Почему S(p) < p даже для π

❓ Вопрос

У двух ноутбуков одинаковое число ядер, а ускорения на π разные. Почему?

✅ Ответ

P/E-ядра, разные турбо-частоты, питание от батареи, нагрев и троттлинг, фоновые процессы. Поэтому в отчёте всегда — **паспорт стенда** и сырые данные.

- В ДЗ0 тот же замер делаете вы: сначала прогноз в `REPORT.md`, потом `make bench`

Разбор чужого кода

Решение с форума: «распараллелить π и измерить ускорение»

```
int nthreads = omp_get_num_threads();      /* количество потоков */
double partial[nthreads];                  /* частичные суммы для каждого потока */
for (int k = 0; k < nthreads; k++) partial[k] = 0.0;

clock_t start = clock();
#pragma omp parallel
{
    int id = omp_get_thread_num();
    for (long i = id; i < n; i += nthreads) {
        double x = (i + 0.5) * step;
        partial[id] += 4.0 / (1.0 + x * x);
    }
}
double sum = 0.0;
for (int k = 0; k < nthreads; k++) sum += partial[k];
double elapsed = (double)(clock() - start) / CLOCKS_PER_SEC;
```

🔍 Разбор чужого кода

Пояснение автора: «Каждый поток накапливает частичную сумму в своём элементе массива, что исключает гонки данных. Время измеряем через `clock()`». Запускаем на 1, 2, 4 потоках: **π правильное**. Значит, программа правильная?

Разбор

1. `omp_get_num_threads()` вне региона вернула **1** → `partial[1]`, шаг `i += 1`
2. Каждый поток проходит **все** n итераций: работы в p раз больше
3. Потоки 1…3 пишут в `partial[1..3]` **за границей массива** на стеке — порча памяти, программа может и упасть
   `-fsanitize=address` → `dynamic-stack-buffer-overflow`
4. Сумма берёт только `partial[0]` → поэтому ответ верный (`make asan` в `code/`)
5. `clock()` суммирует время всех потоков → «стало медленнее» видно, но причина не та

Правильный ответ не доказывает правильность параллельной программы. Проверяем: время по `omp_get_wtime`, санитайзеры (TSan, ASan), вопрос к каждой переменной «она общая или своя?».

Итоги

1. OpenMP = потоки над **общей памятью** + директивы; fork-join; неявный барьер в конце региона
2. Всё, что объявлено **до** региона, — общее. Объявляйте переменные внутри
3. Запись в общую переменную без синхронизации — **гонка**, неопределённое поведение
4. Шаблон: **считаем приватно, объединяем редко**
5. Время — только `omp_get_wtime`, повторы, медиана

Следующий раз: `parallel for`, `reduction`, `schedule` — как сделать то же самое одной строкой и когда «одна строка» ломается.

ДЗ0 — до следующего занятия

`homework/hw0-okruzhenie/README.md`

- Создать свой репозиторий из шаблона курса
- Установить компилятор, собрать `hello` и `pi`
- Собрать **паспорт стенда** `SYSTEM.md` скриптом
- Записать прогноз, затем замерить π на 1…2N потоках через `tools/bench.py`, построить графики
- Добиться зелёного CI

Не собирается? Раздел «Частые проблемы» в `docs/02-okruzhenie.md` и чат курса. Не откладывайте на вечер перед С2.

Попробуйте после занятия (необязательно)

Листок `seminars/s01-vvedenie/praktika.md`, заготовки в `code/practice/`:

1. **Блочное разбиение π** — `practice/pi_practice.c`: поток берёт непрерывный диапазон `[begin, end)`, остаток не потерять
2. **Прогноз и замер** своей версии на 1, 2, 4, … потоках
3. **Симпсон** для $\int_0^3 e^{-x^2}\,dx$ — `practice/integral_practice.c`: почему ответ на 1 и 4 потоках расходится в последних знаках?

Сдавать не нужно. Это разминка перед Д30 и способ проверить, что компилятор работает.